JPA 기초

학습목표
JPA 기초 개념
학과 목록/등록/수정/삭제 구현

목차
1. 배경 지식..... 2
    1) JPA (Java Persistence API)..... 2
    2) JPA Entity 클래스..... 2
    3) JPA Repository 기본 메소드.....3
    4) Optional 클래스..... 4
2. jpa1 프로젝트..... 5
    1) 프로젝트 생성..... 5
    2) ToolchainDownloadFailedException 에러.....7
    3) build.gradle 파일 수정..... 8
    4) application.properties..... 9
    5) Department 엔터티 클래스 구현.....10
    6) Department 리포지토리 구현..... 11
    7) DepartmentEdit.java 모델.....11
    8) Department 컨트롤러 구현..... 12
    9) common.css..... 14
    10) department/list.html..... 15
    11) department/edit.html..... 16
    12) 실행.....17

1. 배경 지식

1) JPA (Java Persistence API)

대표적인 Java ORM 기술은 mybatis와 JPA 이다.
아직까지는 mybatis가 좀 더 많이 사용되지만,
미래의 대세는 JPA 이다.

참고

은행, 대기업, 정부 등 거대한 정보 시스템을 개발하는 분야는
  오래 전에 개발한 시스템을 계속 이어서 개발하기 때문에
  상대적으로 옛날 기술을 많이 사용한다. 예: maven + mybatis + JSP, Vue.js
  이런 종류의 정보 시스템을 개발하는 회사들을 SI 회사라고 부른다.

SNS, 쇼핑몰 등의 서비스 시스템은 상대적으로 크기가 작기 때문에
  최신 기술을 적극 도입한다. 예: gradle + JPA + thymeleaf, React.js

2) JPA Entity 클래스

mybatis의 DTO 클래스에 해당하는 것이 JPA Entity 클래스이다.

JPA Entity 클래스는, 데이터베이스 테이블의 레코드에 해당하는 Java 클래스이다.
데이터베이스 조회 결과가 자동으로 엔터티 클래스 객체에 채워져 리턴된다.

@Entity

JPA Entity 클래스 앞에 @Entity 어노테이션을 붙여야 한다.

@Id

기본키(primary key)에 해당하는 멤버 변수에 @Id 어노테이션을 붙여야 한다.

@GeneratedValue(strategy = GenerationType.IDENTITY)

기본키가 Auto Increment 필드이거나 Identity 필드인 경우에
이 어노테이션을 붙여야 한다.

3) JPA Repository 기본 메소드

mybatis의 mapper 인터페이스에 해당하는 것이 JPA Repository 인터페이스이다.

```
package net.skhu.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import net.skhu.entity.Department;

public interface DepartmentRepository extends JpaRepository<Department, Integer>  {

}
```

위와 같은 형태로 DepartmentRepository 인터페이스를 선언하면,
Department 엔터티 클래스 이름
Integer primary key의 타입

이 인터페이스를 구현한 클래스를 spring JPA 가 자동으로 구현해 준다.
이렇게 자동으로 구현된 클래스에는 아래와 같은 기본 메소드가 포함되어 있다.

departmentRepository.findAll()

department 테이블에서 전체 레코드를 조회해서 리턴한다.

departmentRepository.findById(id)

기본키 id 값으로 department 테이블에서 레코드를 조회해서 리턴한다.

departmentRepository.save(department)

Department 객체를 department 테이블에 저장한다.
Department 객체의 id(기본키) 속성값이 0 이면 INSERT 되고, 0 이 아니면 UPDATE 된다.

departmentRepository.deleteById(id)

id(기본키) 값과 일치하는 레코드가 삭제된다.

departmentRepository.count()

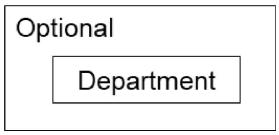
department 테이블의 전체 레코드 수를 리턴한다.

위 메소드들은 Spring JPA가 자동으로 구현해준 메소드이다.

4) Optional 클래스

```
Optional<Department> department = departmentRepository.findById(id);
```

findById 메소드는 department 테이블에서 기본키 id 값으로 레코드를 조회한다.
  이 메소드를 Department 클래스 객체를 바로 리턴하지 않고,
  상자 역할을 하는 Optional 클래스 객체 안에 Department 객체를 넣어서 리턴한다.
  즉 Optional<Department> 클래스 타입의 객체가 리턴된다.



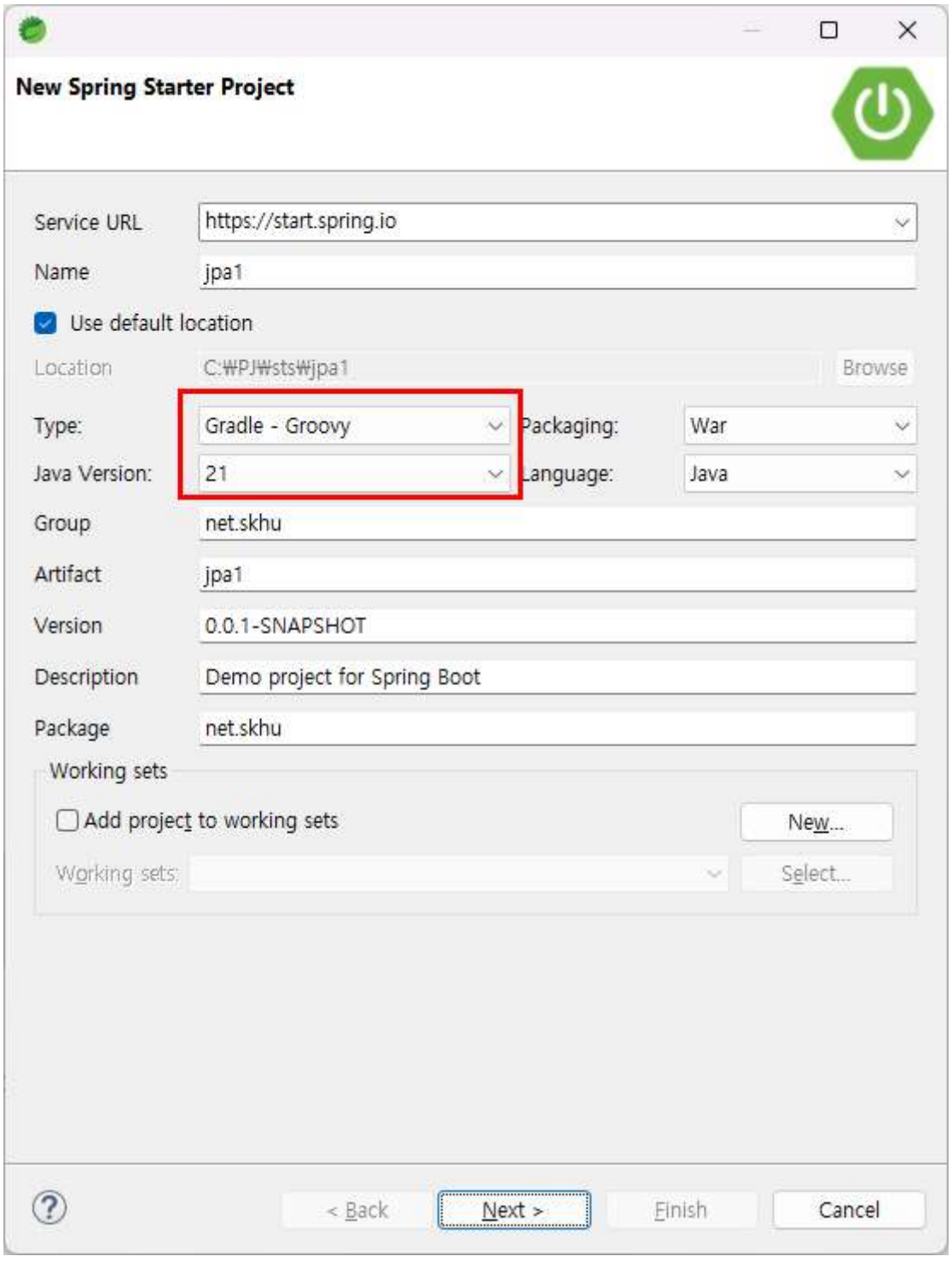
  이 Optional 클래스의 get 메소드를 호출하면, 안에 들어있는 Department 객체가 리턴된다.

  상자 역할을 하는 Optional 클래스를 사용하는 이유는,
  Java 언어의 단점이라고 지적되는 null 오류를 피하기 위해서다.
   (google에서 "java optional null" 키워드로 검색해 보자)

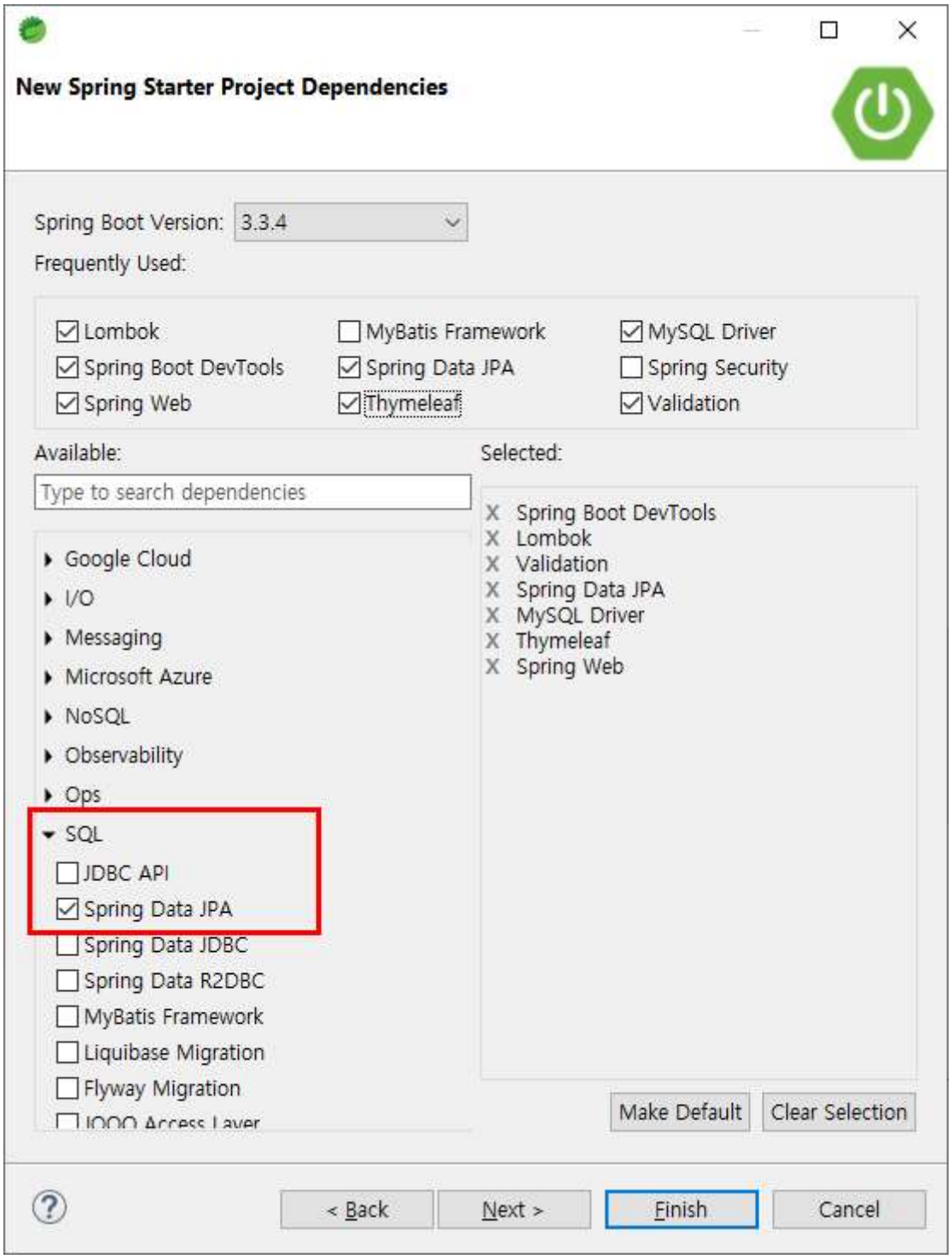
2. jpa1 프로젝트

1) 프로젝트 생성

메뉴: File - New - Spring Starter Project



Name	jpa1
Type	Gradle - Groovy
Packaing	war
Java version	21
Package	net.skhu

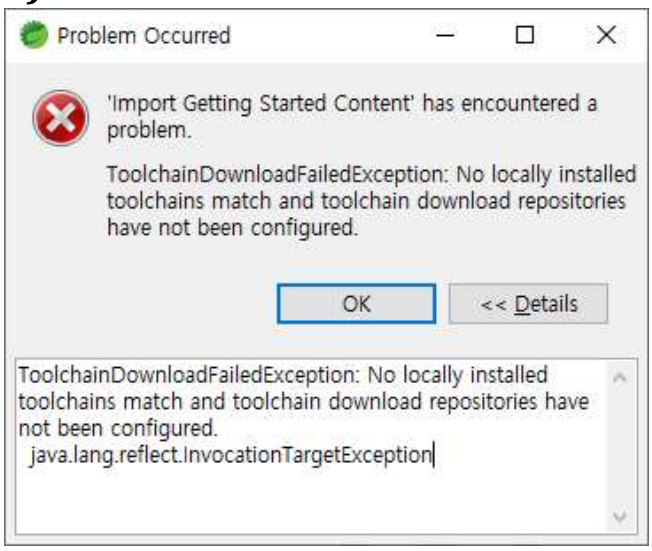


Spring Boot DevTools
Lombok
Validation

SQL 항목 아래
  Spring Data JPA
  MySQL Driver

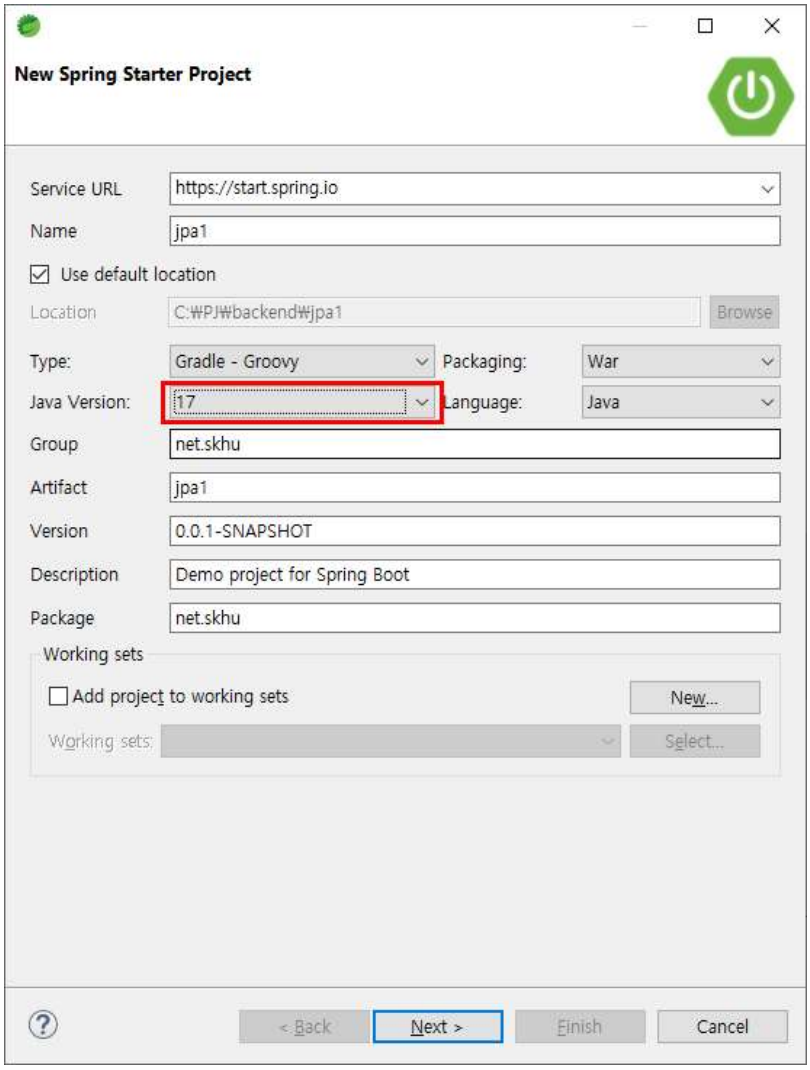
Thymeleaf
Spring Web

2) ToolchainDownloadFailedException 에러



gradle 프로젝트를 생성할 때 위와 같은 에러가 발생할 수도 있다.

이것은 Java 버전 불일치 오류이다.
PC에 설치된 Java 버전과
Spring Boot 프로젝트 생성할 때 지정한 Java 버전이 불일치



PC에 설치된 Java 버전이 17 이라면, 플젝을 만들 때도 17을 선택하자.

Java 버전을 동일하게 맞추었는데도 에러가 계속된다면, Java 17을 삭제하고 Java 21을 설치하자.

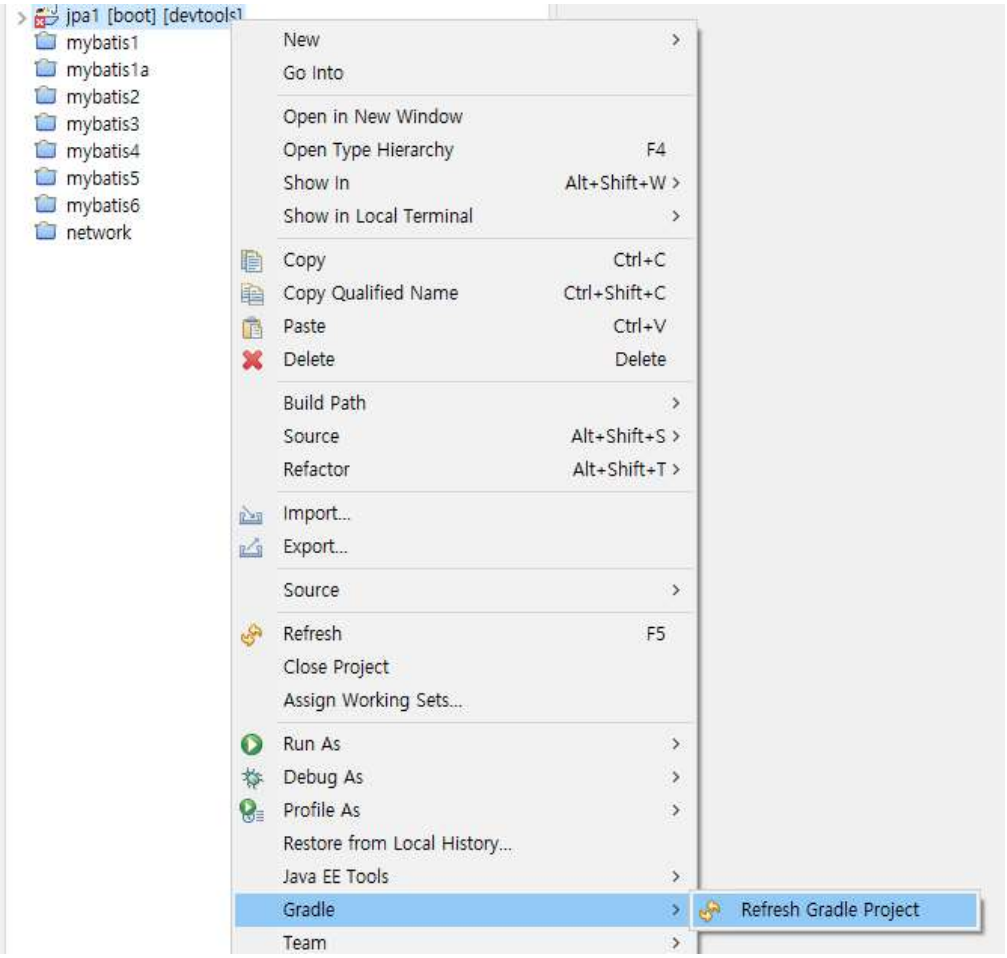
3) build.gradle 파일 수정

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    compileOnly 'org.projectlombok:lombok'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    runtimeOnly 'com.mysql:mysql-connector-j'
    annotationProcessor 'org.projectlombok:lombok'
    providedRuntime 'org.springframework.boot:spring-boot-starter-tomcat'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    implementation 'org.modelmapper:modelmapper:3.2.5'
}
```

gradle 프로젝트의 설정 파일인 build.gradle 파일에
  modelmapper maven dependency 한 줄을 추가하자.
  위의 노란색 한 줄만 추가하면 된다.

maven 프로젝트의 설정 파일: pom.xml
gradle 프로젝트의 설정 파일: build.gradle

Referesh Gradle Project



build.gradle 파일을 수정한 후에는 반드시 Refresh Gradle Project 해야 한다.

4) application.properties

```
spring.application.name=jpa1
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/student2?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=user1
spring.datasource.password=skhuA+4.5
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=TRACE
logging.level.net.skhu=DEBUG
server.port=8088
```

참고:

아래 내용은 나중에 JPA에 조금 익숙해 진 다음에 읽어도 된다.

```
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

엔터티 클래스의 studentNo 속성에 자동으로 연결될 데이터베이스 필드 명이
studentNo 형태라면 위 설정이 필요하다. (camel case)
student_no 형태이면 위 설정이 필요없다. (snake case)

camel case의 예: departmentManagerOfficeNumber
snake case의 예: department_manager_office_number

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

JPA 엔진에 의해서 자동 생성되고 실행되는 SQL 명령을 실시간으로 이클립스 콘솔창에 보여준다.

5) Department 엔터티 클래스 구현

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;
}
```

JPA Entity 클래스는, 데이터베이스 테이블의 레코드에 해당하는 Java 클래스이다.
데에터베이스 조회 결과가 자동으로 엔터티 클래스 객체에 채워져 리턴된다.

@Data

Lombok 어노테이션이다.
Lombok에 의해서 getter, setter, equals, hashCode 메소드가 자동으로 구현된다.

@Entity

JPA Entity 클래스 앞에 @Entity 어노테이션을 붙여야 한다.

@Id

기본키(primary key)에 해당하는 멤버 변수에 @Id 어노테이션을 붙여야 한다.

@GeneratedValue(strategy = GenerationType.IDENTITY)

기본키가 Auto Increment 필드이거나 Identity 필드인 경우에
이 어노테이션을 붙여야 한다.

6) DepartmentRepository.java 리포지토리 구현

src/main/java/net/skhu/repository/DepartmentRepository.java

```java
package net.skhu.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import net.skhu.entity.Department;

public interface DepartmentRepository extends JpaRepository<Department, Integer>  {

}
```

`Integer` 이 부분은 primary key의 타입이다.

department 테이블의 id 필드 타입은 int 이지만,
Java Generic 문법에서 < > 괄호 안에 int 같은 기본 자료형을 적을 수 없으니,
Integer 같은 클래스라고 적어야 한다.

7) DepartmentEdit.java 모델

src/main/java/net/skhu/model/DepartmentEdit.java

```java
package net.skhu.model;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import lombok.Data;

@Data
public class DepartmentEdit {
    int id;

    @NotEmpty @NotBlank
    String name;

    @NotEmpty @NotBlank
    String shortName;

    @NotEmpty @NotBlank
    String phone;
}
```

8) DepartmentController.java 구현

src/main/java/net/skhu/controller/DepartmentController.java

```java
package net.skhu.controller;

import java.util.List;
import java.util.Optional;
import org.modelmapper.ModelMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.server.ResponseStatusException;
import jakarta.validation.Valid;
import lombok.extern.slf4j.Slf4j;
import net.skhu.entity.Department;
import net.skhu.model.DepartmentEdit;
import net.skhu.repository.DepartmentRepository;

@Controller
@Slf4j
@RequestMapping("department")
public class DepartmentController {

    @Autowired DepartmentRepository departmentRepository;
    ModelMapper modelMapper = new ModelMapper();

    @GetMapping("list")
    public String list(Model model) {
        List<Department> departments = departmentRepository.findAll();
        model.addAttribute("departments", departments);
        return "department/list";
    }

    @GetMapping("edit")
    public String edit(Model model, @RequestParam("id") int id) {
        Optional<Department> department = departmentRepository.findById(id);
        if (department.isEmpty())
            throw new ResponseStatusException(HttpStatus.NOT_FOUND);
        DepartmentEdit departmentEdit = modelMapper.map(department.get(), DepartmentEdit.class);
        model.addAttribute("departmentEdit", departmentEdit);
        return "department/edit";
    }

    @PostMapping(value="edit", params="cmd=save")
    public String edit(Model model, @Valid DepartmentEdit departmentEdit, BindingResult bindingResult) {
        try {
            log.debug(departmentEdit.toString());
            if (bindingResult.hasErrors())
                throw new Exception("저장할 수 없습니다");

            Department department = modelMapper.map(departmentEdit, Department.class);
            departmentRepository.save(department);
            return "redirect:list";
        }
        catch (Exception ex) {
            bindingResult.reject("", null, ex.getMessage());
            return "department/edit";
        }
    }

    @PostMapping(value="edit", params="cmd=delete")
     public String delete(Model model, @Valid DepartmentEdit departmentEdit, BindingResult bindingResult)
{
        try {
            departmentRepository.deleteById(departmentEdit.getId());
            return "redirect:list";
        }
        catch (Exception ex) {
            bindingResult.reject("", null, ex.getMessage());
            return "department/edit";
        }
    }

    @GetMapping("create")
    public String create(Model model) {
        DepartmentEdit departmentEdit = new DepartmentEdit();
        model.addAttribute("departmentEdit", departmentEdit);
        return "department/edit";
    }

    @PostMapping("create")
```

```
85      public String create(Model model, @Valid DepartmentEdit departmentEdit, BindingResult bindingResult)
86  {
87          try {
88              log.debug(departmentEdit.toString());
89              if (bindingResult.hasErrors())
90                  throw new Exception("저장할 수 없습니다");
91
92              Department department = modelMapper.map(departmentEdit, Department.class);
93              departmentRepository.save(department);
94              return "redirect:list";
95          }
96          catch (Exception ex) {
97              bindingResult.reject("", null, ex.getMessage());
98              return "department/edit";
99          }
      }
  }
```

```
Optional<Department> department = departmentRepository.findById(id);
                                // id 값으로 department를 조회해서 리턴한다.

if (department.isEmpty())  // 조회 결과 Optional 상자 안에 department 객체가 없으면
     throw new ResponseStatusException(HttpStatus.NOT_FOUND);
        // 404 Not Found Exception을 throw 한다.


departmentRepository.save(department);
  JPA repository의 save 메소드는 INSERT/UPDATE 양쪽 다 사용할 수 있다.
  department 객체의 id(기본키) 속성값이 0 이면 INSERT 되고, 0 보다 크면 UPDATE 된다.
```

9) common.css

src/main/resources/static/common.css

```
div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
.shadow { box-shadow: 2px 2px 5px gray; }

.btn { padding: 0.4em 1em; border: 1px solid gray;
       border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
       text-decoration: none; color: black;
       display: inline-block; }
.btn:active {
      -ms-transform: translateY(2px);
      -webkit-transform: translateY(2px);
      transform: translateY(2px);
      background: #ccc; }

table.list { border-collapse: collapse; width: 100%; }
table.list td { padding: 4px; border: 1px solid gray; }
table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }

div.row { margin-bottom: 0.8em; }
div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }

input[type=text], input[type=number] { padding: 0.3em; }
select { padding: 0.3em; }
h1 { margin: 0.5em 0; }
hr { margin: 1em 0; }
.error { color: red; padding: 0.2em; margin-left: 1em; }

div.pagination table { border-collapse: collapse; margin-top: 1em; }
div.pagination table td { border: 1px solid #ccc; text-align: center; }
div.pagination table td.active { background-color: #48f; }
div.pagination table a { text-decoration: none; color: blue; display: inline-block;
                         width: 25px; padding: 6px; }
div.pagination table td.active a { color: white; }
```

10) department/list.html

src/main/resources/templates/department/list.html

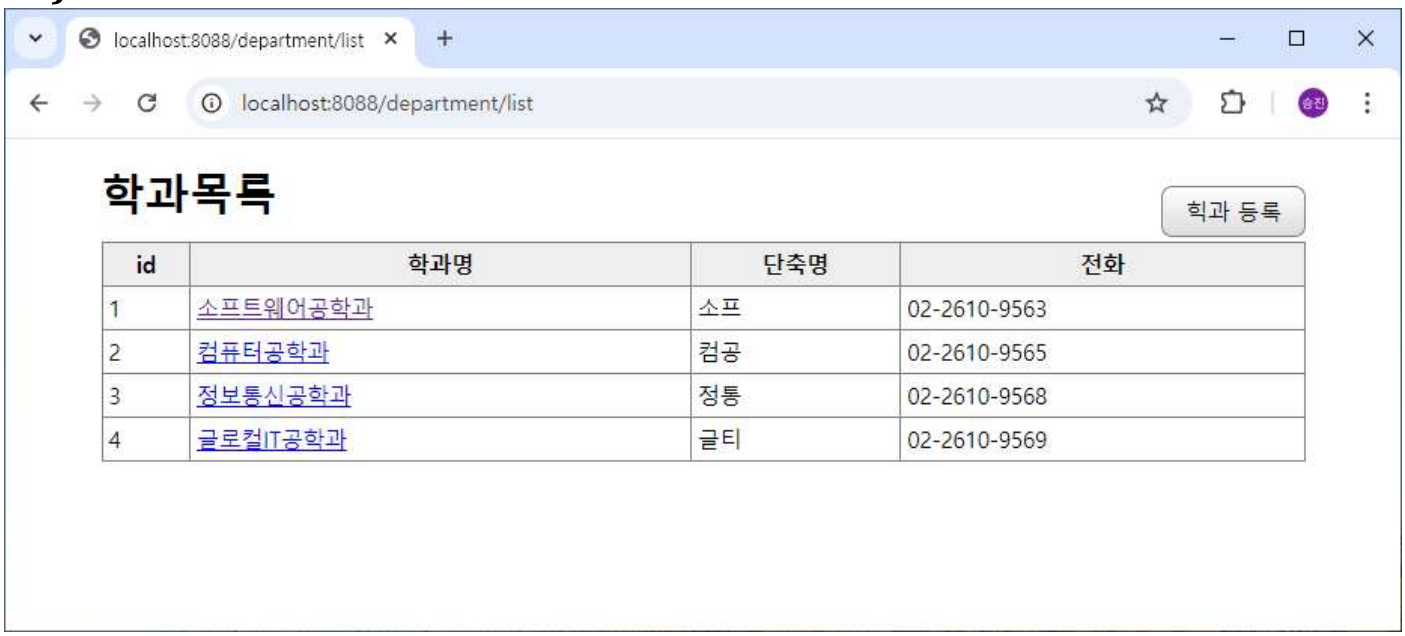
```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <script type="text/javascript" th:src="@{/common.js}"></script>
  <style>
    a.btn { float:right; margin-top: -37px; }
  </style>
</head>
<body>
<div class="container">
  <h1>학과목록</h1>
  <a href="create" class="btn">학과 등록</a>
  <table class="list">
    <thead>
      <tr>
        <th>id</th>
        <th>학과명</th>
        <th>단축명</th>
        <th>전화</th>
      </tr>
    </thead>
    <tbody>
      <tr th:each="dp : ${departments}">
        <td th:text="${dp.id}"></td>
        <td>
            <a th:text="${ dp.name }" th:href="@{ edit(id=${ dp.id }) }"></a>
        </td>
        <td th:text="${dp.shortName}"></td>
        <td th:text="${dp.phone}"></td>
      </tr>
    </tbody>
  </table>
</div>
</body>
</html>
```

11) department/edit.html

src/main/resources/templates/department/edit.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <style>
    form { width: 500px; padding: 1em 3em; }
    button:nth-of-type(1) { margin-left: 1em; }
  </style>
</head>
<body>
<div class="container">
 <form method="post" th:object="${departmentEdit}" class="shadow">
    <h1 th:if="${departmentEdit.id > 0}">학과 수정</h1>
    <h1 th:if="${departmentEdit.id == 0}">학과 등록</h1>
    <div class="row">
      <label>학과명:</label>
      <input type="text" th:field="*{name}" />
      <div class="error" th:errors="*{name}"></div>
    </div>
    <div class="row">
      <label>단축명:</label>
      <input type="text" th:field="*{shortName}" />
      <div class="error" th:errors="*{shortName}"></div>
    </div>
    <div class="row">
      <label>전화:</label>
      <input type="text" th:field="*{phone}" />
      <div class="error" th:errors="*{phone}"></div>
    </div>
    <hr />
    <button type="submit" class="btn"
             name="cmd" value="save">저장</button>
    <button type="submit" class="btn"
             th:if="${ departmentEdit.id > 0 }"
             onclick="return confirm('삭제하시겠습니까?')"
             name="cmd" value="delete">삭제</button>
    <a href="list" class="btn">목록으로</a>
    <span class="error" th:errors="${departmentEdit}"></span>
  </form>
</div>
</body>
</html>
```

12) 실행


localhost:8088/department/list
localhost:8088/department/list
학과목록
학과 등록
id
학과명
단축명
전화
1
소프트웨어공학과
소프
02-2610-9563
2
컴퓨터공학과
컴공
02-2610-9565
3
정보통신공학과
정통
02-2610-9568
4
글로컬IT공학과
글티
02-2610-9569